

Salesforce CRM Project Documentation

WhatNext Vision Motors

Salesforce CRM Implementation Documentation

Name : Aswitha B

College : R.M.K Engineering College

1. Project Overview

WhatNext Vision Motors is an internal Salesforce application built to support the day-to-day operations of an automotive dealership network. Rather than tracking vehicles, customers, dealers, orders, test drives and service work in separate spreadsheets or disconnected tools, the org brings all of this information into a single connected workspace.

At the center of the application is the sales cycle: a customer expresses interest in a vehicle, takes a test drive, and eventually places an order. Before an order can move forward, the system checks that the vehicle is actually in stock and routes the order to a suitable dealer automatically, removing a step that would otherwise be handled manually by staff.

Two background processes reduce manual follow-up: a reminder is generated ahead of a scheduled test drive, and orders that were previously blocked by a stock shortage are re-evaluated once inventory is replenished. On top of this data foundation, a Lightning app gives staff a single place to navigate between records, and reports/dashboards give managers visibility into overall activity.

2. Objectives

The build was guided by five goals:

1. Bring vehicle, customer, dealer, order, test-drive and service data into one Salesforce org instead of scattered records.
2. Give staff a repeatable, guided way to create and track vehicle orders that takes stock levels into account before confirming a sale.
3. Cut down on repetitive manual steps — such as assigning a dealer or sending a test-drive reminder — through Flow and Apex automation.
4. Catch bad or incomplete data at the point of entry using validation rules and Apex checks, rather than after the fact.
5. Present the whole system through an organized Lightning app so users are not hunting across menus to find the record type they need.

3. Phase 1 — Requirements and Planning

Before any objects were built, the underlying business scenario was translated into a concrete set of Salesforce requirements. This phase covered requirement gathering, defining the scope of the build, sketching the data model, and outlining who should be able to see what.

3.1 Business Requirements

Looking at how an automotive dealer actually operates — before a sale, during a sale, and after a sale — the following core requirements were identified:

- Keep a record of every vehicle available for sale, including how many units are in stock.
- Maintain a profile for each customer that can be reused across test drives, orders and service visits.
- Track the dealers who fulfil orders, along with their contact and location details.
- Capture every vehicle order placed, with enough detail to know its current status.
- Link each order back to the customer, the vehicle and the dealer involved, so nothing is orphaned or duplicated.

3.2 Scope of the Build

Area	What it covers
Vehicle Management	Model, pricing, on-hand stock, and availability status for each vehicle.
Dealer Management	Dealer identity, location and contact details used to fulfil orders.
Customer Management	Customer contact details and preferences used across sales, test drives and service.
Orders & Test Drives	Recording purchase orders and scheduling test drives, each tied to a customer and a vehicle.
Service Requests	Logging post-sale service needs and linking them to the relevant customer and vehicle.

3.3 Data Model

The object model reflects how these entities relate to one another in real life: a Vehicle can appear on many Orders and Test Drives, a Customer can be linked to several Orders, Test Drives and Service Requests, and a Dealer can be tied to multiple vehicle transactions. Modelling these as lookup relationships — rather than repeating the same details on every record — keeps the data consistent and makes it easy to jump from one related record to another.

The screenshot shows the Salesforce Object Manager interface. At the top, there's a navigation bar with 'Setup', 'Home', and 'Object Manager' (selected). Below this, a breadcrumb trail reads 'SETUP > OBJECT MANAGER'. The main heading is 'Vehicle Order'. On the left, a sidebar menu lists various configuration options: 'Details', 'Fields & Relationships' (highlighted), 'Page Layouts', 'Lightning Record Pages', 'Buttons, Links, and Actions', 'Compact Layouts', 'Field Sets', and 'Object Limits'. The main content area is titled 'Vehicle Order New Relationship' and includes a 'Help for this Page' link. It shows 'Step 2. Choose the related object'. There are 'Previous', 'Next', and 'Cancel' buttons. Below this, a text prompt says 'Select the other object to which this object is related.' A dropdown menu labeled 'Related To' is open, showing 'Vehicle Customer' as the selected option. At the bottom, there are again 'Previous', 'Next', and 'Cancel' buttons.

Building the lookup relationship from Vehicle Order to Vehicle Customer in Object Manager — this is how the order, vehicle, dealer and customer records stay connected.

3.4 Security Model

Access control was planned around Salesforce's standard layers rather than a single blanket setting:

- Profiles — the baseline permissions assigned to each user.
- Roles and role hierarchy — used to control which records a user can see based on their position in the organization.
- Permission sets — used to grant extra access on top of a profile, without changing the profile itself.
- Object- and field-level permissions — fine-grained control over what each user type can view or edit.

4. Phase 2 — Backend Build: Objects, Automation and Apex

With the plan in place, this phase covered the actual configuration work: building the custom objects and fields, adding validation, and writing the automation and Apex logic behind the order process.

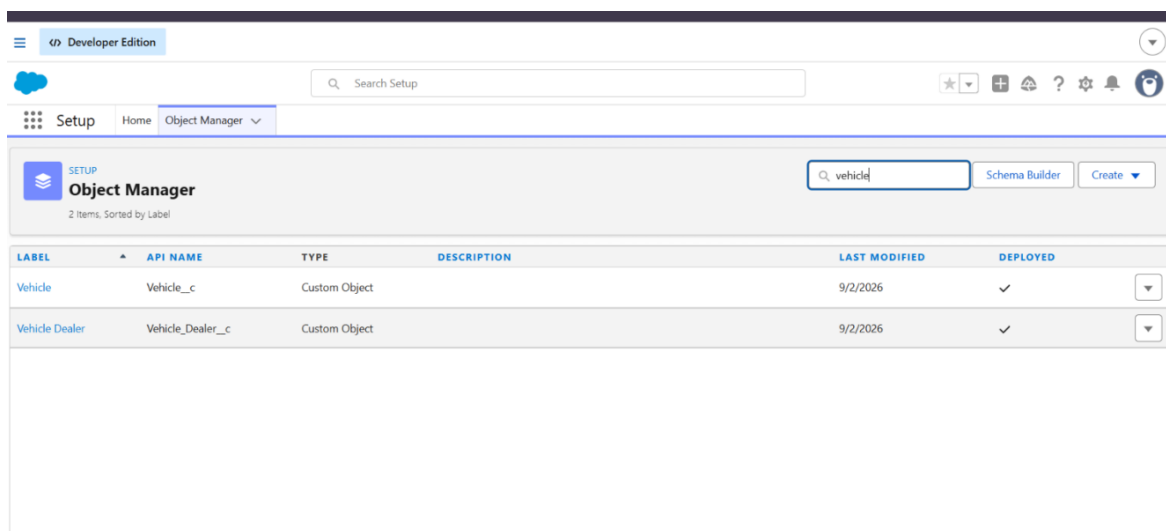
4.1 Environment

All configuration was carried out in Salesforce Lightning Experience, using the standard setup tools for objects, fields, relationships, Flow, Apex, reports and security — no external development environment was required for this phase.

4.2 Custom Objects

Six custom objects form the backbone of the application:

- Vehicle — model, price, stock quantity, availability and associated dealer.
- Dealer — dealer name, location, city and contact information.
- Customer — contact details used for purchases, test drives and service visits.
- Vehicle Order — links a customer, vehicle and dealer, and tracks order status.
- Vehicle Test Drive — links a customer and vehicle, with an appointment date and status.
- Vehicle Service Request — links a customer and vehicle, with an issue description and status.



LABEL	API NAME	TYPE	DESCRIPTION	LAST MODIFIED	DEPLOYED
Vehicle	Vehicle_c	Custom Object		9/2/2026	✓
Vehicle Dealer	Vehicle_Dealer__c	Custom Object		9/2/2026	✓

Object Manager, filtered to "vehicle", showing two of the custom objects created for the app (Vehicle and Vehicle Dealer).

4.3 Fields

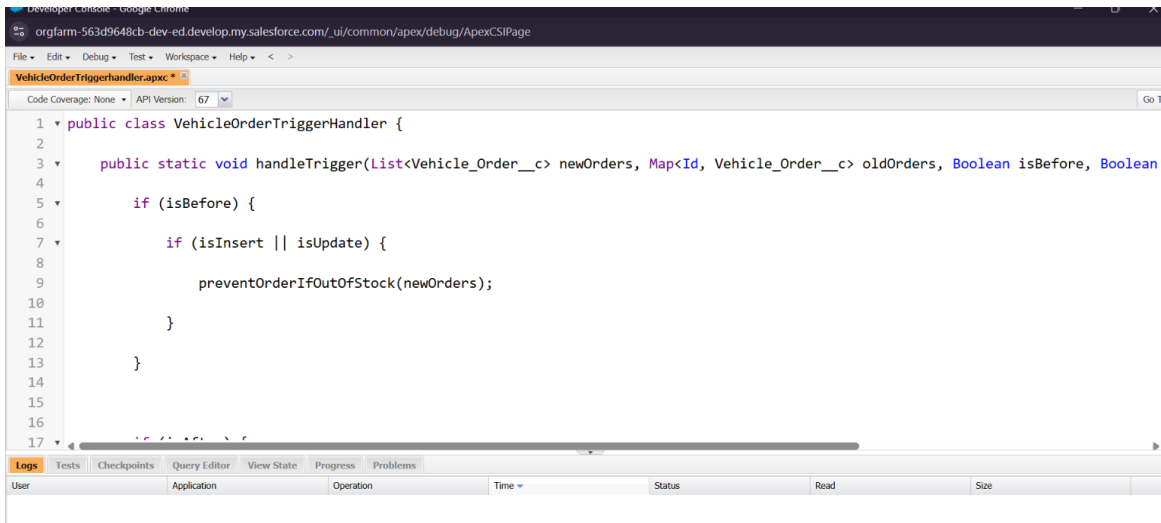
Each object carries the fields needed to support its role in the process — for example, the Vehicle object tracks stock quantity and status so the order process has something concrete to check against, while the Order object stores lookups to the Customer, Vehicle and Dealer records plus an order date and status picklist.

The screenshot shows the Salesforce Setup interface. At the top, there is a search bar labeled 'Search Setup'. Below it, the navigation bar includes 'Setup', 'Home', and 'Object Manager'. The 'Object Manager' tab is selected, and the breadcrumb trail shows 'SETUP > OBJECT MANAGER'. The main heading is 'Vehicle Order'. On the left, a sidebar lists various configuration options: 'Details', 'Fields & Relationships' (which is highlighted), 'Page Layouts', 'Lightning Record Pages', 'Buttons, Links, and Actions', 'Compact Layouts', 'Field Sets', and 'Object Limits'. The main content area shows the configuration for the 'Status' field. The 'Field Label' is 'Status'. Under the 'Values' section, the option 'Enter values, with each value separated by a new line' is selected. Below this, a text area contains the following values: 'Pending', 'Confirmed', 'Delivered', and 'Canceled'. At the bottom of the configuration area, there are two checkboxes: 'Display values alphabetically, not in the order entered' (which is unchecked) and 'Use first value as default value' (which is also unchecked). At the bottom of the screen, there is an 'Einstein' icon and a close button.

Creating the Status picklist field on Vehicle Order, with Pending, Confirmed, Delivered and Canceled as the available values.

4.4 Validation Rule — Stock Check

A validation rule stops a Vehicle Order from being confirmed if the linked Vehicle record shows zero units in stock. This keeps an obviously invalid sale from being recorded and forces the order to stay in a pending state until stock is available.

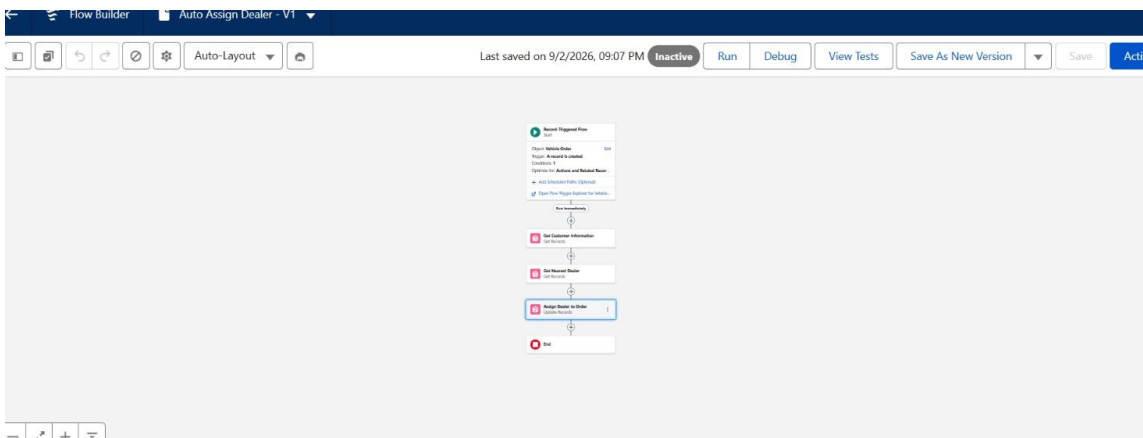


VehicleOrderTriggerHandler — the preventOrderIfOutOfStock() check runs before an order is inserted or updated, which is what actually enforces the stock rule.

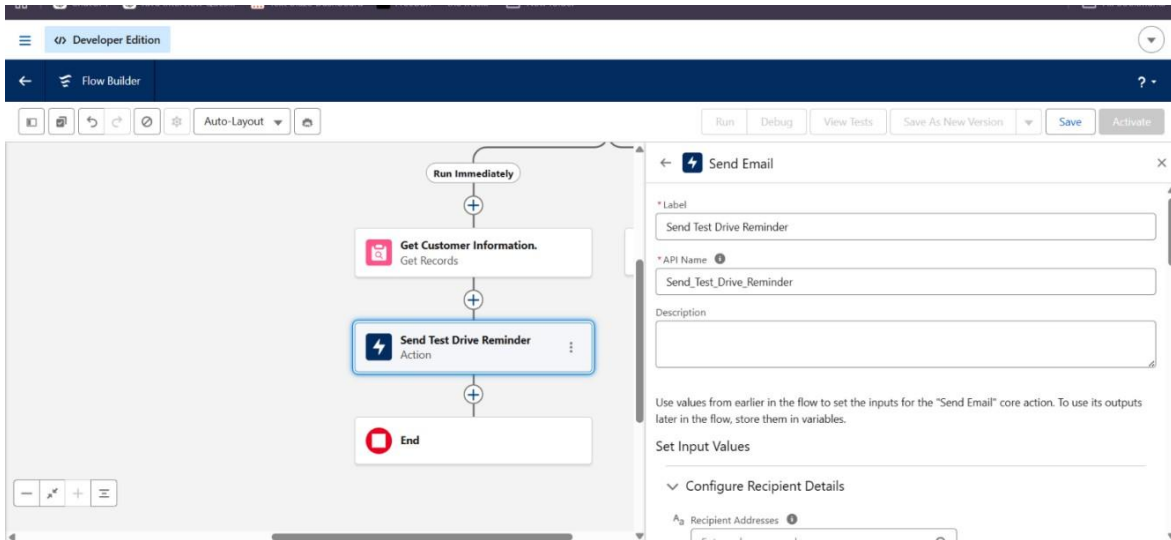
4.5 Flow Automation

Two record-triggered flows remove manual steps from the process:

- Dealer Assignment — when a new order is created, the flow looks up the customer and an appropriate dealer, then updates the order with that dealer automatically.
- Test Drive Reminder — an email reminder is sent to the customer ahead of their scheduled test-drive date, without a staff member needing to send it by hand.



Auto Assign Dealer — a record-triggered flow on Vehicle Order that looks up the customer and nearest dealer, then updates the order automatically.



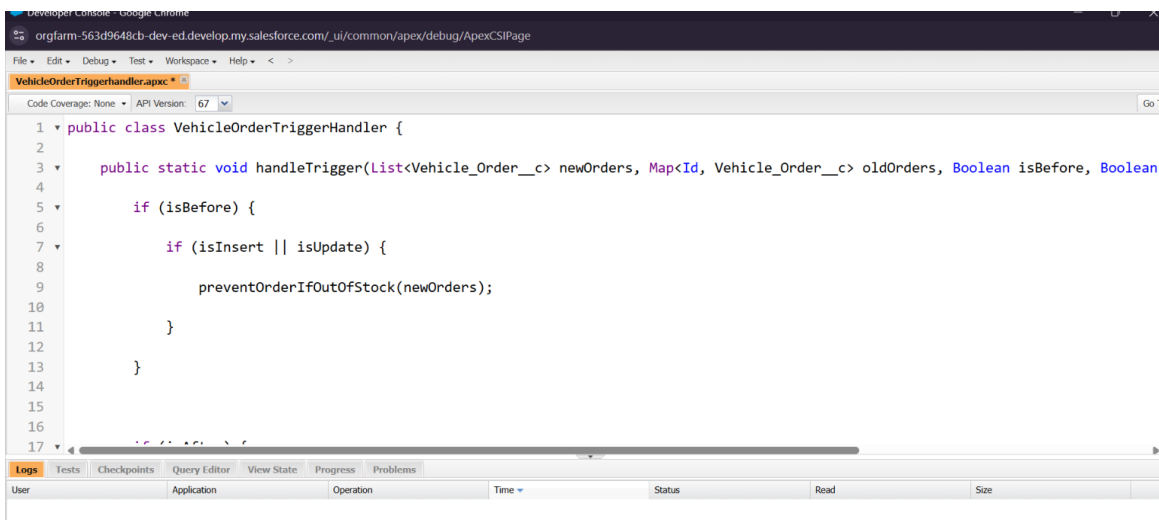
The Send Test Drive Reminder action inside the test-drive flow, configured as a Send Email step.

4.6 Apex Classes and Trigger

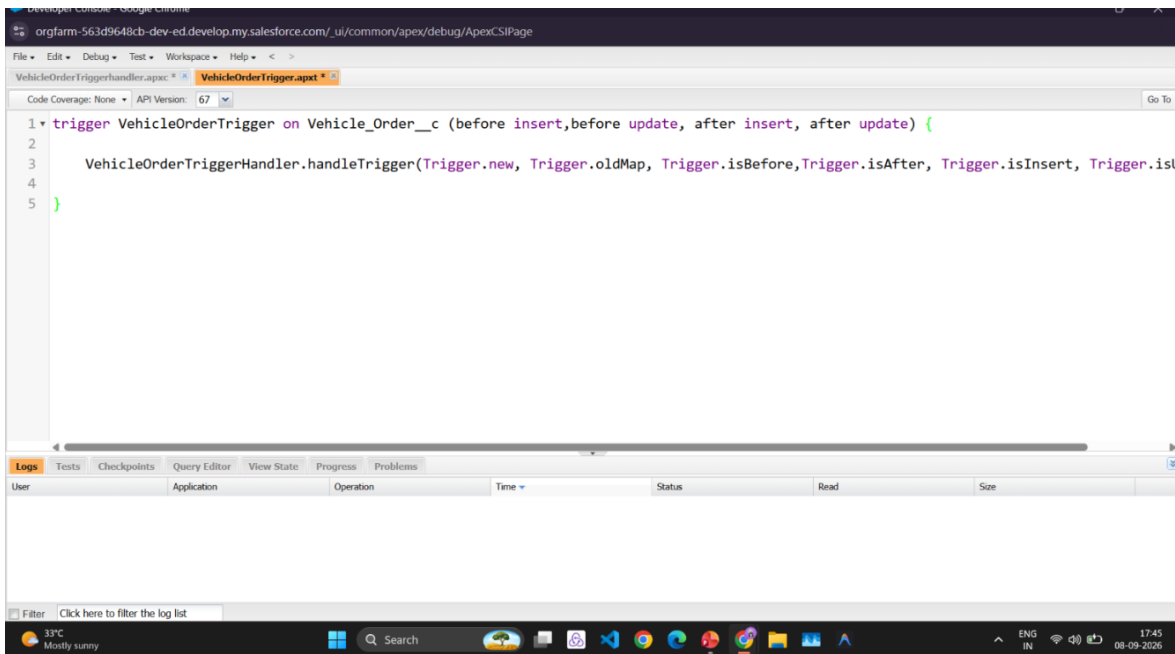
Where declarative tools were not enough on their own, an Apex trigger on Vehicle Order (before insert/update) delegates to a handler class. The handler is responsible for:

- Checking the stock quantity on the related vehicle before allowing an order to be confirmed.
- Blocking the operation with a clear error when stock is insufficient.
- Reducing the vehicle's stock count once an order is confirmed.

Keeping the logic in a separate handler class — rather than writing it directly inside the trigger — makes the code easier to read, test and extend later.



VehicleOrderTriggerHandler.apxc in the Developer Console, showing the handleTrigger method delegating to preventOrderIfOutOfStock.



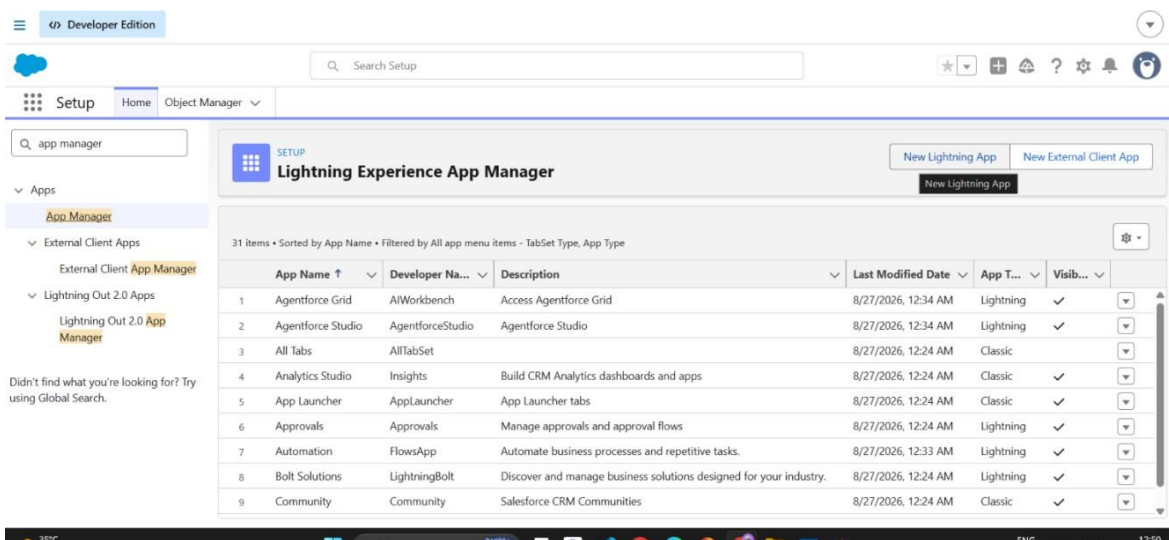
VehicleOrderTrigger.apxt — the trigger itself simply hands off to the handler class for before/after insert and update.

5. Phase 3 — User Interface and Experience

This phase focused on how the system feels to use day-to-day, building on top of the objects and automation from Phase 2.

5.1 Lightning App

A single Lightning app named WhatNext Vision Motors brings together the Vehicles, Dealers, Customers, Vehicle Orders, Test Drives and Service Requests tabs, so staff do not need to search for these objects individually in Setup or the App Launcher.



Lightning Experience App Manager, where the WhatNext Vision Motors app was created alongside the org's other apps.

The New Lightning App wizard — naming the app and setting its navigation-bar color.

5.2 Page Layouts and Dynamic Forms

Page layouts were arranged so the most relevant fields for each object appear first, with related lists (such as a vehicle's orders and test drives) placed underneath. Dynamic Forms were used on records where certain fields only make sense to show under specific conditions, rather than displaying every field regardless of context.

5.3 User Management

Users were set up with the profiles and access levels appropriate to their role in the dealership, so that record creation — such as adding a new customer — matches the permissions granted through the security model described in Phase 1.

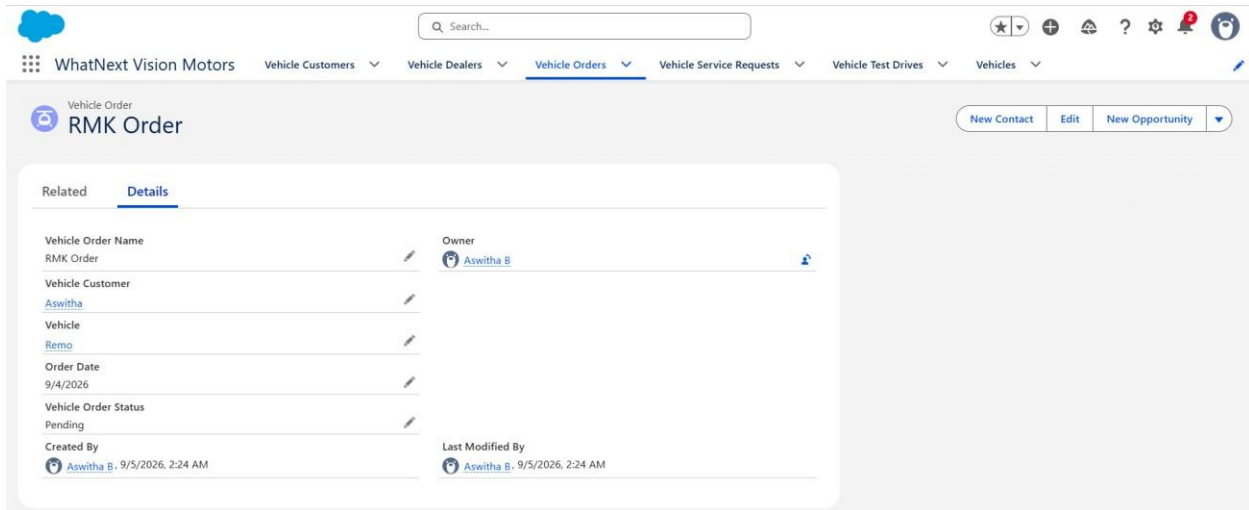
New Vehicle Customer Entry Screen

5.4 Reports and Dashboards

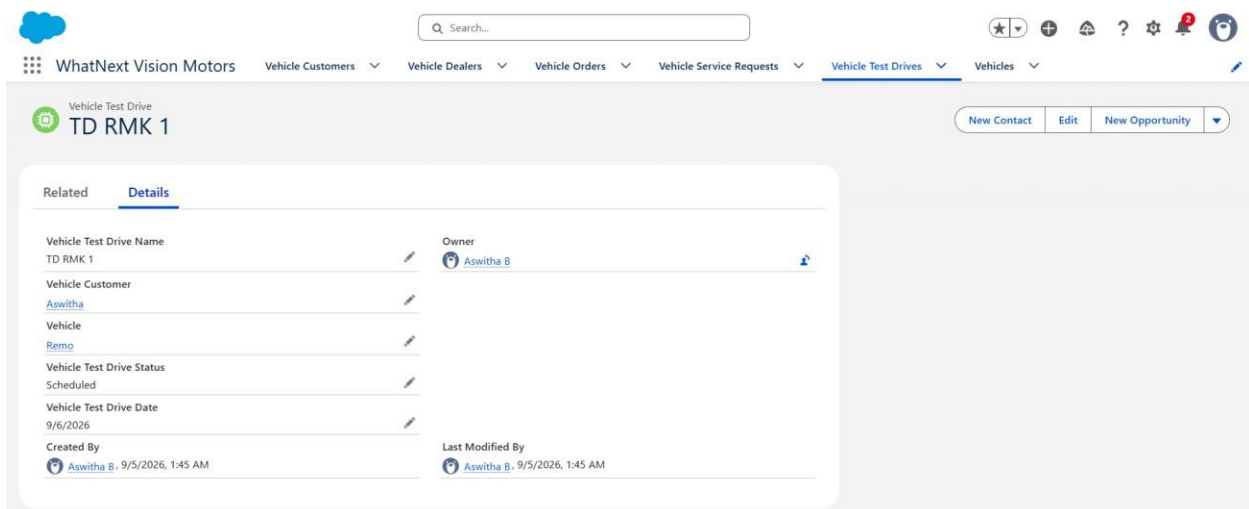
Standard reports summarize activity such as orders by status or test drives by date, and a dashboard built on top of these reports gives a quick, visual read on how the business is trending without opening individual records.

5.5 Lightning Record Pages

Individual record pages were configured with a highlights panel, key details, and related lists — for example, a Vehicle record page surfaces its linked orders and test drives directly, so a user does not need to run a separate search to see how that vehicle is being used.



Vehicle Order Lightning Record Page Showing Key Details and Record Relationships



Lightning Record Page Displaying Vehicle Test Drive Details and Record Relationships

6. Phase 4 — Data, Testing and Security

With the interface in place, this phase covered loading realistic data, configuring org-wide security, and testing the automation built in earlier phases.

6.1 Data Loading

Sample records were prepared and loaded in a controlled sequence so that relationships would resolve correctly:

1. Prepare sample data for each object.
2. Map each data field to its corresponding Salesforce field.
3. Load the records, starting with objects that have no dependencies (such as Dealer and Vehicle).
4. Confirm the records were created without errors.
5. Spot-check that lookups between related records — for example, an Order pointing to the correct Customer and Vehicle — resolved as expected.

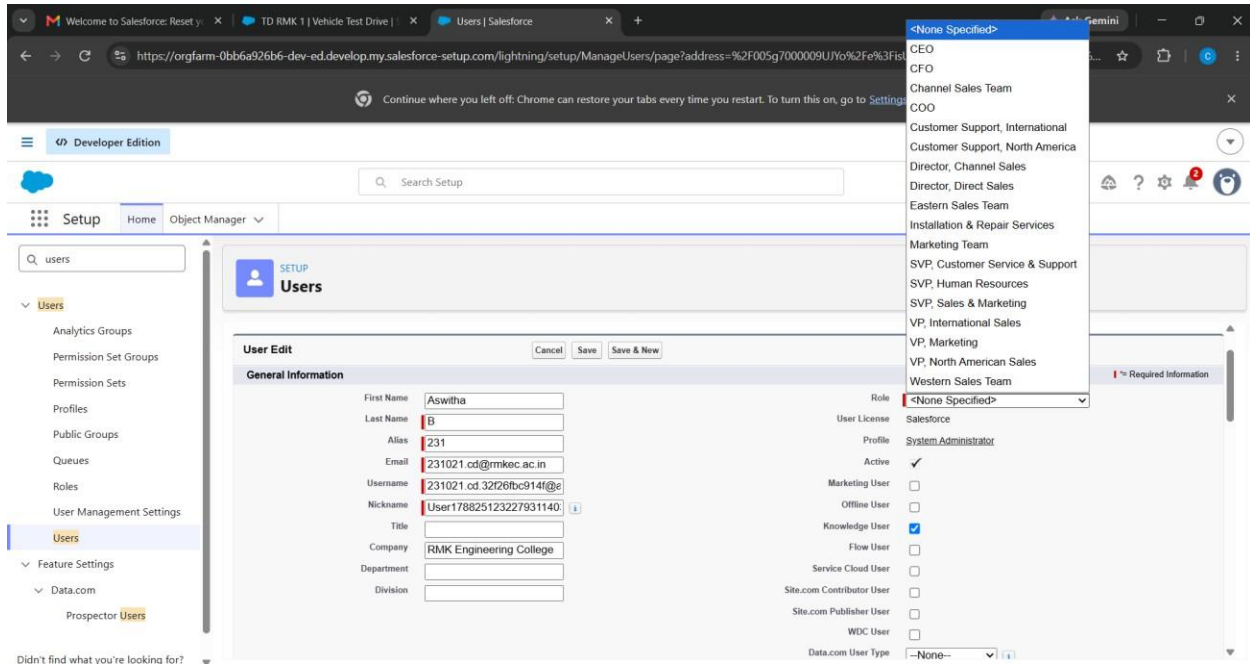
Depending on the object and the number of records involved, either the Data Import Wizard or Data Loader was used.

6.2 Profiles

Profiles set the baseline of what each user type can do — which objects, fields and features are visible to them by default.

6.3 Roles and Role Hierarchy

A role hierarchy reflecting the dealership's reporting lines (for example, CEO above Sales Manager above Sales Representative) determines how records roll up for visibility purposes, so a manager can see the records owned by their team without needing separate sharing rules for every case.



Dealership Role Hierarchy Showing CEO, Manager, and Representative Reporting Structure

6.4 Permission Sets

Where a subset of users needed access beyond their base profile — for instance, additional object permissions for staff handling service requests — permission sets were used instead of creating extra profiles, keeping the profile list itself easier to manage.

6.5 Sharing Rules

Sharing settings were configured so each user can access the records relevant to their role while records outside their remit stay restricted, in line with the org-wide defaults and the role hierarchy described above.

6.6 Test Cases

Each automated feature was tested with concrete input data and the resulting output was captured. The four test scenarios below illustrate the approach; each was executed in the org with screenshots attached separately.

Test Case 1 — Creating a Vehicle Record

Goal: confirm that a valid vehicle can be saved with all required details.

Field	Value
Vehicle Name	Remo
Model	SUV
Stock Quantity	1
Dealer	Dealer 2

The screenshot displays the 'Vehicle Record' page for a vehicle named 'Remo'. The page is part of the 'WhatNext Vision Motors' application. The top navigation bar includes links for 'Vehicle Customers', 'Vehicle Dealers', 'Vehicle Orders', 'Vehicle Service Requests', and 'Vehicle Test Drives'. The main content area shows the vehicle details under the 'Details' tab. The details include: Vehicle Name (Remo), Vehicle Model (SUV), Price (blank), Stock Quantity (1), Dealer (Dealer 2), and Status (Out of Stock). The 'Created By' field shows 'Aswitha B' on 9/5/2026 at 1:44 AM, and the 'Last Modified By' field shows 'Aswitha B' on 9/9/2026 at 9:49 PM.

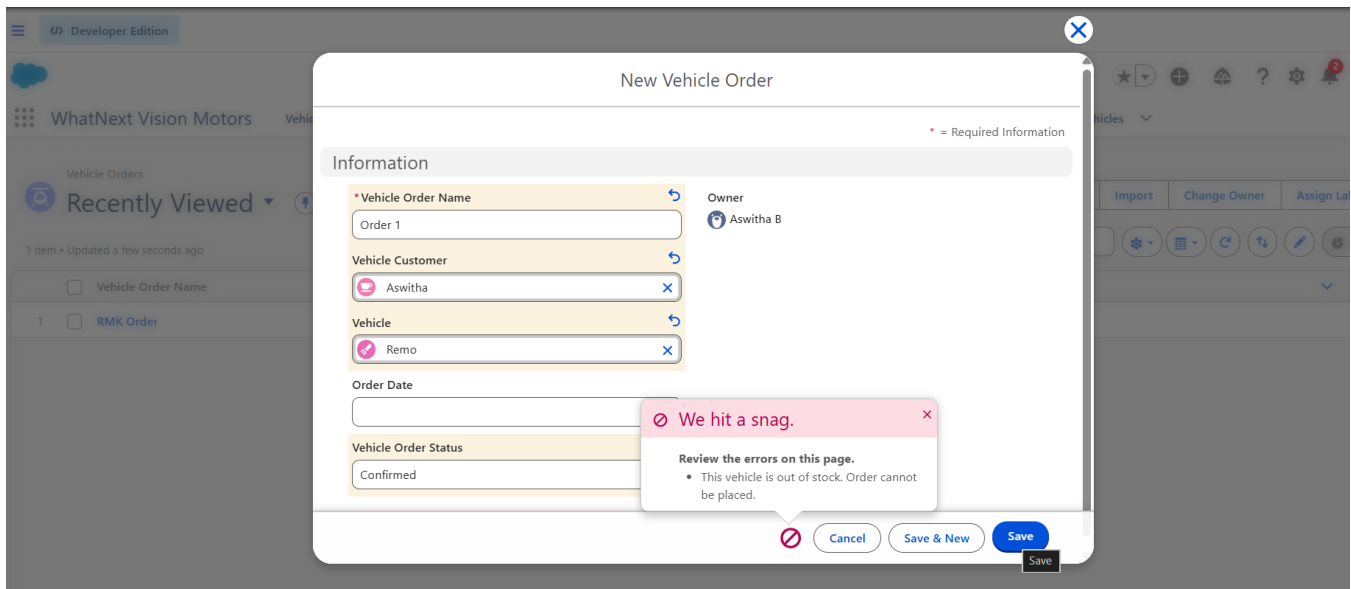
Vehicle Record Page Displaying Vehicle Details and Dealer Information

Test Case 2 — Blocking an Out-of-Stock Order

Goal: confirm the validation rule stops an order from being confirmed once the linked vehicle's stock reaches zero.

The screenshot displays the 'Vehicle Record' page for a vehicle named 'Remo'. The page is part of the 'WhatNext Vision Motors' application. The top navigation bar includes links for 'Vehicle Customers', 'Vehicle Dealers', 'Vehicle Orders', 'Vehicle Service Requests', and 'Vehicle Test Drives'. The main content area shows the vehicle details under the 'Details' tab. The details include: Vehicle Name (Remo), Vehicle Model (SUV), Price (blank), Stock Quantity (0), Dealer (Dealer 2), and Status (Out of Stock). The 'Created By' field shows 'Aswitha B' on 9/5/2026 at 1:44 AM, and the 'Last Modified By' field shows 'Aswitha B' on 9/10/2026 at 12:03 AM.

Vehicle Out of Stock quantity

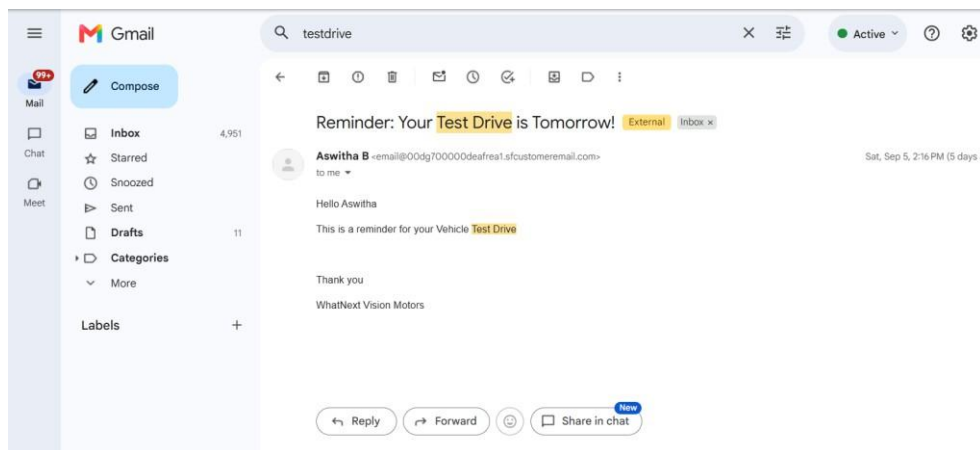


Vehicle Order could not be placed because of out of stock quantity

Test Case 3 — Test Drive Reminder Email

Goal: confirm the reminder flow fires correctly ahead of a scheduled test drive.

- Customer: an existing customer record
- Customer email: a valid, reachable address
- Status: Scheduled
- Test drive date: set to the following day



Test-drive reminder email received in the customer's inbox

Test Case 4 — Reprocessing a Pending Order

Goal: confirm that an order previously blocked by insufficient stock can proceed once stock is replenished

Developer Edition

WhatNext Vision Motors

Vehicle Customers

Vehicle Dealers

Vehicle Orders

Vehicle Service Requests

Vehicle Test Drives

Vehicles

Vehicle Remo

New Contact

Edit

Related

Details

Vehicle Name

Remo

Vehicle Model

SUV

Price

Stock Quantity

10

Dealer

Dealer 2

Status

Available

Created By

Aswitha B. 9/5/2026, 1:44 AM

Owner

Aswitha B.

Last Modified By

Aswitha B. 9/10/2026, 12:10 AM

Vehicle Remo stock is replenished

WhatNext Vision Motors

Vehicle Customers

Vehicle

Vehicle Order "Order 1" was created.

Vehicle Test Drives

Vehicles

Vehicle Order Order 1

New Contact

E

Related

Details

Vehicle Order Name

Order 1

Vehicle Customer

Aswitha

Vehicle

Remo

Order Date

Vehicle Order Status

Confirmed

Created By

Aswitha B. 9/10/2026, 12:11 AM

Owner

Aswitha B.

Last Modified By

Aswitha B. 9/10/2026, 12:11 AM

New vehicle order is placed after vehicle Remo stock is replenished

7.Phase 5 — Deployment and Ongoing Maintenance

The final phase covers how the build moves between orgs, how it is kept healthy afterward, and how issues get resolved when they come up.

7.1 Deployment Strategy

The following metadata components make up the deployable unit for this project and move together between environments using Salesforce's standard deployment tooling (such as Change Sets or a metadata-based deployment):

- Custom objects and fields
- Object relationships
- Validation rules
- Flows
- Apex classes and triggers
- Lightning pages and the Lightning app
- Reports and dashboards

7.2 Monitoring and Maintenance

A few standard Salesforce monitoring tools are relied on to keep the automation healthy after go-live:

- Apex Jobs — to confirm scheduled or batch Apex is running as expected.
- Debug Logs — for digging into unexpected Apex or automation behaviour.
- Flow error emails — to catch failures in the dealer-assignment or reminder flows early.
- Periodic access reviews — to make sure users still hold only the permissions their role requires.

7.3 Troubleshooting Approach

When something does not behave as expected, the same sequence is followed:

1. Identify the problem — pin down which object, flow, or trigger is producing the wrong result.
2. Reproduce it — repeat the scenario with controlled test data so it isn't a one-off.
3. Review the configuration — check the relevant fields, validation rule, flow or Apex logic.
4. Check the logs — pull debug logs or flow error details where automation is involved.
5. Fix the issue — update the configuration or code responsible.
6. Retest — rerun the original scenario to confirm the fix actually resolves it.
7. Deploy the fix — move the corrected metadata to the target org through the standard deployment process.

8. Conclusion

The WhatNext Vision Motors build brings vehicle, customer, dealer, order, test-drive and service information into a single Salesforce org, replacing what would otherwise be a set of disconnected records. Stock is checked automatically before an order is confirmed, dealers are assigned without manual intervention, and customers receive test-drive reminders without staff needing to send them individually.

Beyond the automation, the Lightning app and record pages give staff one consistent place to work, the security model keeps access aligned with each user's role, and the reports/dashboards give managers a way to see overall activity at a glance. The test cases documented above provide evidence that each of these pieces behaves as intended.

Taken together, the project shows how a combination of Salesforce's declarative tools — objects, validation rules, Flow — and a small amount of targeted Apex can support a real business process end to end.